IST 387: Introduction to Applied Data Science
Lab 9: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- A classification Example
- Supervised Learning Via Support Vector Machines
- Support Vector Machines in R
- Supervised Learning Via Classification and Regression Trees
- Important R functions

A Classification Example

Let's create a supervised learning model to classify whether a person's income exceeds $50,000 a year. This is a classification problem, because we are restricting the outcome to two discrete choices (≤$50,000 or >$50,000).

You might be thinking, why don't we just predict the actual value for income?

That would be a regression problem, rather than a classification problem.

We extracted the dataset from the UCI (University of California, Irvine) Machine Learning repository. This repository is a collection of almost 500 datasets that are freely available as a service to the machine learning community. You can explore the information about the dataset for yourself from the UCI website: http://archive.ics.uci.edu/ml/datasets/Adult.

As shown in the table below, there are 15 columns (14 attributes, and the known target variable "income").

Column Name	Possible Values
income	≤50K, >50K
age	Continuous
workclass	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked
fnlwgt	Continuous
education	Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool
education-num	Continuous
marital-status	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse
occupation	Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
relationship	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
race	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex (gender)	Female, Male
capital-gain	Continuous
capital-loss	Continuous
hours-per-week	Continuous
native-country	United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South Korea, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Colombia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong-Kong, Holland-Netherlands

While many of the variables are self-explanatory, some are trickier to understand.

- The attribute **fnlwgt** represents final weight. While these weights are important for obtaining accurate estimates of population values, we will ignore them for this analysis just to keep things simple.

- The attribute **education_num** stands for the number of years of education in total.

- The attribute **relationship** represents the responding person's role in the family.

- Finally, the **capital _gain** and **capital _loss** attributes are income from investment sources other than wages or salary.

Keeping all that in mind, we can now load the data with the following command by first reading the dataset from the UCI website. Specifically, the dataset is at:
http://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data

```
library(readr)
webSite <- "http://archive.ics.uci.edu/ml/"
dataLocation <- "machine-learning-databases"
adultData <- "/adult/adult.data"
theUrl <- paste0(webSite, dataLocation, adultData)
incomeDF <- read_csv(theUrl, col_names=FALSE)
```

Because the UCI dataset does not have metadata within the datafile, let's assign column names to the data frame.

```
col_names <- c("age", "workclass", "fnlwgt", "education", "education_num",
               "marital_status", "occupation", "relationship", "race",
               "sex", "capital_gain", "capital_loss",
               "hours_per_week", "native_country",
               "income")
colnames(incomeDF) <- col_names
```

We can now inspect the dataset:

```
install.packages("tidyverse")
library("tidyverse")
glimpse(incomeDF)
```

The output is as follows:

```
> glimpse(incomeDF)
Rows: 32,561
Columns: 15
$ age            <dbl> 39, 50, 38, 53, 28, 37, 49, 52, 31, 42, 37, 30, 23, 32,…
$ workclass      <chr> "State-gov", "Self-emp-not-inc", "Private", "Private", …
$ fnlwgt         <dbl> 77516, 83311, 215646, 234721, 338409, 284582, 160187, 2…
$ education      <chr> "Bachelors", "Bachelors", "HS-grad", "11th", "Bachelors…
$ education_num  <dbl> 13, 13, 9, 7, 13, 14, 5, 9, 14, 13, 10, 13, 13, 12, 11,…
$ marital_status <chr> "Never-married", "Married-civ-spouse", "Divorced", "Mar…
$ occupation     <chr> "Adm-clerical", "Exec-managerial", "Handlers-cleaners",…
$ relationship   <chr> "Not-in-family", "Husband", "Not-in-family", "Husband",…
$ race           <chr> "White", "White", "White", "Black", "Black", "White", "…
$ sex            <chr> "Male", "Male", "Male", "Male", "Female", "Female", "Fe…
$ capital_gain   <dbl> 2174, 0, 0, 0, 0, 0, 0, 0, 14084, 5178, 0, 0, 0, 0, 0, …
$ capital_loss   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0…
$ hours_per_week <dbl> 40, 13, 40, 40, 40, 40, 16, 45, 50, 40, 80, 40, 30, 50,…
$ native_country <chr> "United-States", "United-States", "United-States", "Uni…
$ income         <chr> "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "…
```

The output shows that the data structure has 32,561 rows and 15 columns.

Let's explore some more:

```
table(incomeDF$income)
```

The output is as follows:

```
<=50K  >50K
24720  7841
```

The output shows us that there are 24,720 people that were classified as having an income less than 50K, and 7,841 people that had an income of more than 50k.

We need to clean the data to get it ready for analysis. First, we will ignore the **fnlwgt** attribute, which we can do by removing it from the dataframe with the **select( )** command from the dplyr package.

```
library(dplyr)
incomeDF <- dplyr::select(incomeDF, -fnlwgt)
```

Next, we can create better predictor variables by creating more general groupings for categorical variables such as **education**. This is known as ***feature engineering***, which is the process of applying human knowledge to the data to create better predictor variables that can help the machine learning algorithms create more accurate results and/or work faster. We can also use feature engineering to help handle missing data.

As part of the feature engineering process, we also decide what to do with missing elements. In this dataset, missing elements are noted with a "?" We could try to replace those unknown values via a process of *imputing* those missing values. Imputing is a fancy word for using statistics to make an educated guess. For example, a simple approach could be to calculate the mean of the known values or the most commonly occurring value and use that value for all instances that are missing. If we removed rows that contained the missing data, we might be losing important aspects of the data.

The code below does the feature engineering by first changing the strings into factors (categorical variables) to make our fixes easier:

```
#update each column to be a factor - if its a string
incomeDF <- mutate_if(incomeDF, is.character, factor)
```

We will make some repairs to the **workclass**, **education**, **marital_ status**, **occupation**, and **native_country** variables. In each case we will recode the original set of seven or more different options into a much smaller set of four to six options using the **levels( )** command. For several variables, we will also grab the "?" which was the indicator for missing data, and recode that into a level called unknown:

```
levels(incomeDF$workclass) <- list(public=c("Federal-gov","State-gov",
                          "Local-gov"), private=c("Private", "Self-emp-not-inc",
                                        "Self-emp-inc"),
              other= c("Never-worked","Without-pay", "?"))

levels(incomeDF$education) <- list(noCollege = c("Preschool", "1st-4th",
                  "5th-6th", "7th-8th", "9th", "Assoc-acdm","Assoc-voc","10th",
                  "11th","12th", "HS-grad"),
      graduate=c("Bachelors", "Some-college"),
      master = c("Masters", "Prof-school"),
      phd = c("Doctorate"))

levels(incomeDF$marital_status) <- list(
  divorce=c("Divorced","Seperated"),
  married = c("Married-AF-spouse",
              "Married-civ-spouse",
              "Married-spouse-absent"),
  notmarried = c("Never-married",
  widowed = c("Widowed"))
```

```
levels(incomeDF$occupation) <- list(
  clerical=c("Adm-clerical"),
  tradeskill=c("Craft-repair",
                "Machine-op-inspct", "Other-service"),
  physical = c("Transport-moving",
                "Priv-house-serv", "Protective-serv",
                "Handlers-cleaners"),
  highskill = c("Sales", "Tech-support", "Armed-Forces",
                 "Prof-speciality", "Exec-managerial"),
  agricultr = c("Farming-fishing"),
  unknown = c("?"))
```

```
levels(incomeDF$native_country) <- list(
  Asia=c("Vietnam", "Laos", "Cambodia", "Thailand",
         "Japan", "South", "China", "India", "Hong-Kong",
         "Iran", "Philippines", "Taiwan"),
  NorthAmerica = c("Canada", "Cuba",
                   "Dominican-Republic", "Guatemala", "Haiti",
                   "Honduras", "Jamaica", "Mexico", "Nicaragua",
                   "Puerto-Rico", "El-Salvador", "United-States",
                   "Outlying-US(Guam-USVI-etc)"),
  SouthAmerica = c("Ecuador", "Peru", "Colombia",
                   "Trinidad&Tobago"),
  Europe=c("England", "France", "Germany", "Greece",
           "Holland-Netherlands", "Italy", "Hungary",
           "Ireland", "Poland", "Portugal", "Scotland",
           "Yugoslavia"),
  unknown = c("?")
)
```

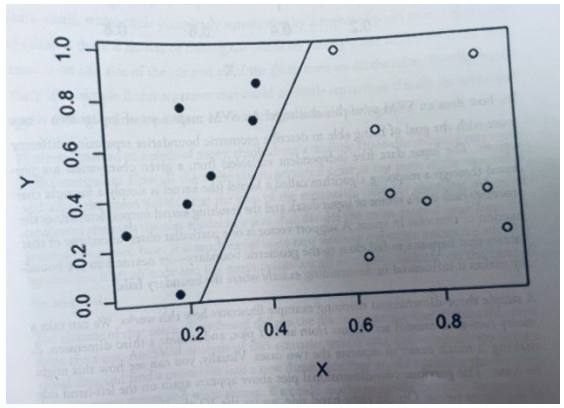
There is no one correct way to do this kind of recoding— you can use your knowledge of the problem domain as a starting point and be ready to revise your choices if analytic results suggest that you should.

SUPERVISED LEARNING VIA SUPPORT VECTOR MACHINES

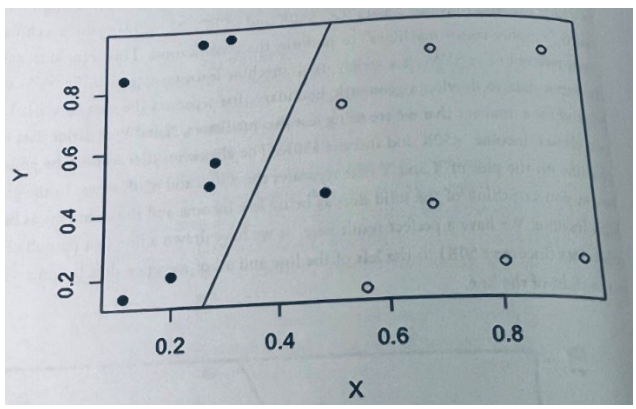
Using this dataset, lets now create a model to classify people into low or high income. The algorithm needs to use the predictor to divide the data into two classes (i.e. ≤50K and >50K). We will use a technique called "**support vector machines**" to perform this classification.

Let's pretend for a moment that we are using just two predictors, X and Y, to divide data into two classes: income ≤50K and income >50K. The algorithm tries to find the position of line on the plot of X and Y that separates the ≤50K and >50K cases.

In the graph below, you can think of the solid dots as being low income and the open dots as being high income. We have a perfect result here, as we have drawn a line that puts all the solid dots (income ≤50K) to the left of the line and all the open dots (income >50K) to the right of the line.



It is more typical, however, that the data are not this cleanly divided. In another dataset shown on the graph below, we have one solid dot that ends up on the wrong side of the line. The algorithm could try to adjust the position of the line to get a clean separation between the solid and open dots, bur real data are usually way too messy to achieve a perfect result.



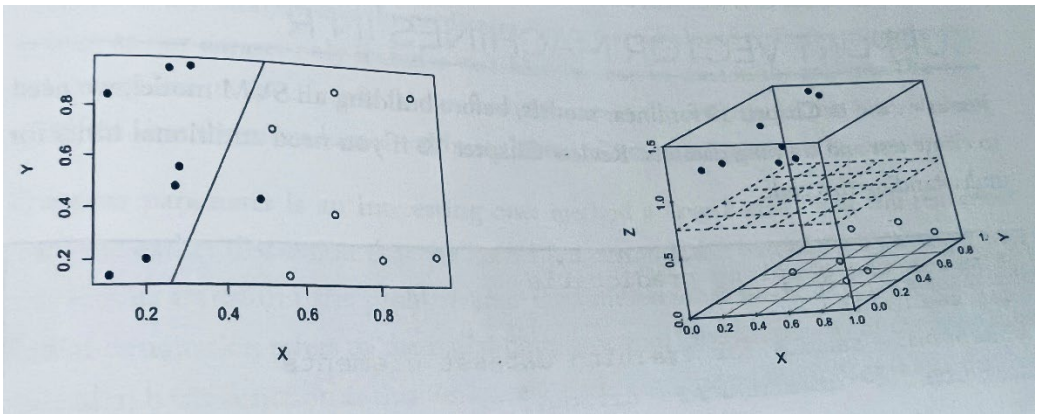
So, how does an SVM solve this challenge? An SVM maps a set of inputs into a new space with the goal of being able to describe geometric boundaries separating different regions.

The input data from a given observation are processed through a mapping algorithm called a **kernel** (the kernel is simply a formula that processes each case's vector of input data), and the resulting kernel output determines the position of that case in space.

The **support vectors** are observations of the dataset that fall close to the geometric boundary—their nearness to the boundary make them influential in determining exactly where the boundary falls.

A simple three-dimensional mapping example illustrates how this works. We can take a messy two-dimensional set of dots from an X-Y plot, and include a third dimension, Z, making it much easier to separate the two cases. The previous two-dimensional plot above appears again on the

left-hand side of the figure below. On the right-hand side, we lay the 2D plot flat on the ground and then add a third dimension, Z. With this third dimension we can now draw a plane that perfectly separates the solid dots from the open dots. Our kernel in this case would turn the X and Y inputs into Z and each dot's position on Z puts it above or below the separator plane.



SUPPORT VECTOR MACHINES IN R

Before building an SVM model, we need to create **test** and **training datasets**.

```
#makes the sampling predictable
set.seed(111)
#Randomly sample for training dataset elements
library("caret")
trainList <- createDataPartition(
      y=incomeDF$income, p=0.7, list=FALSE)

#create train and test datasets
trainData <- incomeDF[trainList,]
testData <- incomeDF[-trainList,]
```

So now we have two separate datasets, representing the training and test breakdown of the original data. Remember, because this is a random set of indices, if the code were to run again without using **set.seed(111),** we would get different train and test datasets.

We are now ready to train our support vector model. First, we will build the model using the simple partitioning approach, with separate datasets for training and testing. The following command generates a model based on the training dataset:

```
svm.model <- train(income ~ ., data = trainData,
              method = "svmRadial",
              trControl = trainControl(method ="none"),
              preProcess = c("center", "scale"))
```

Let's examine this command in some detail:

The first argument, **income ~.,** specifies the model we want to test using R's formula language. Using the word "**income**" as the first element in this expression means that we want to use the

binary **income** variable (i.e., whether the person earns more or less than 50K) as the outcome variable that our model predicts.

The **tilde character (~)** is an R expression that separates the left-hand side of the formula (output) from the right-hand side inputs.

Finally, the **dot character (.)** is a shorthand that tells R to use all the other variables in the data frame as the predictors.

The **data parameter** lets us specify which data frame to use in the analysis. In this case, we have specified that the procedure should use **the trainData** training set that we developed.

The next parameter is an interesting one: **method = "svmRadial."** The **svmRadial** designation refers to the **radial basis function**. In simplified terms, the radial basis function kernel takes the set of inputs from each row in a dataset and calculates a distance value based on the combination of the many variables in the row. The weighting of each data point is adjusted by the algorithm to get the maximum separation of distance between the income ≤50K cases and the income >50K cases.

The **trControl** argument refers to how we should control the training of the model, but since we are using a simple partitioning (train and test) approach, we do not have to worry about that just yet, which is why we use the method of "none."

The final parameter states that there should be some pre-processing of the data. Specifically, for some machine learning algorithms, including SVM, the results may improve when each variable is adjusted to have a mean of zero and a standard deviation of one. So, we center and scale the data before the SVM algorithm starts. The "**center**" transformation calculates the mean for an attribute and subtracts it from each value. The "**scale**" transformation calculates the standard deviation for an attribute and divides each value by that standard deviation.

Let's have a look at what our output structure contains:

```
svm.model
```

The output is as follows:

```
Support Vector Machines with Radial Basis Function Kernel

19886 samples
   14 predictor
    2 classes: '<=50K', '>50K'

Pre-processing: centered (36), scaled (36)
Resampling: None
```

The real test for our SVM is to use the model we generated through this training process to predict the outcomes in a novel dataset.

The following commands use testData to evaluate our model using a confusion matrix:

```
testData <- na.omit(testData)
predictValues <- predict(svm.model, newdata = testData)
confusionMatrix(predictValues, testData$income)
```

The output is as follows:

```
Confusion Matrix and Statistics

          Reference
Prediction <=50K >50K
     <=50K  6452  982
     >50K    243  770

               Accuracy : 0.855
                 95% CI : (0.8473, 0.8624)
    No Information Rate : 0.7926
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.4776

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9637
            Specificity : 0.4395
         Pos Pred Value : 0.8679
         Neg Pred Value : 0.7601
             Prevalence : 0.7926
         Detection Rate : 0.7638
   Detection Prevalence : 0.8801
      Balanced Accuracy : 0.7016

       'Positive' Class : <=50K
```

The command in the block above uses our model output from before, namely **svm.model**, as the parameter for prediction. It uses **test Data,** which the support vectors have never seen before, to generate predictions. The output from the **predict( )** command is a vector of factors, with two levels "<=50K" and "50K.'

In the second command above, we output the confusion matrix (and many additional statistics) about our prediction. As you can see in the output, the confusion matrix contains four values:

- The top-left value shows the number of correct predictions for "<=50K."
- The lower-right value in the matrix shows the number of correct predictions for ">50K,"
- The top-right value shows the number of predictions of "<=50K" that were not correct (i.e. the SVM model predicted "<=50K" but the actual value was ">50K"), and finally,
- The lower-left value shows the number of incorrect predictions for ">50K".

So, in looking at the confusion matrix, along the off diagonal of the confusion matrix, we see the incorrect classifications: 243 cases that were ≤50K, but were classified as >50K by the SVM and

982 cases that were >50K, but were classified as ≤50K by the SVM. On the main diagonal, we see 6452 cases correctly classified as ≤50K and 770 cases correctly classified as >50K.

Overall, it looks like the model is OK, but far from perfect. The simplest way of judging model quality is to look at the **accuracy** of the model, which can be calculated as:

(6452+770)/ (6452+770+243+982) = 0.855

We also need to consider two additional measures of error.

- **Sensitivity:** The number of correct classifications of ≤50K divided by the total number of classifications of ≤50K:

  6452/ (6452 + 243) = 0.963

  and

- **Specificity**: The number of correct classifications of >50K divided by the total number of classifications of >50K.

  770/(770 + 982) = 0.439

In machine learning, as we add more data, the model tends to get more accurate. However, we might be able to achieve a "good enough" result with a smaller, more manageable dataset. Of course, using more data might allow the model to capture unusual patterns, but these patterns might not generalize to new unknown data. So, before building a model from our large dataset, let's try a smaller training set. We can always go back later and build models on the complete dataset.

In this example, we will use just 5% of the data for our training to see whether there is a difference in model performance. Since we have randomly sampled 5% of the data for training, 95% is left for the test dataset. Remember that we are doing this not because it is an especially good idea, but just to see how much the model's performance is impaired by using a much smaller training dataset. Let's look at the calculated confusion matrix:

```
#set up the same data as for the 1st example
set.seed(111)
#create the test and train dataset
subsetList <- createDataPartition(
  y= incomeDF$income, p=0.05, list=FALSE)
trainData <-incomeDF[subsetList,]
testData <- incomeDF[-subsetList,]
#Generate the SVM model
trainData <- na.omit(trainData)
svm.model <- train(income ~ ., data = trainData,
                   method = "svmRadial",
                   trControl = trainControl(method = "none"),
                   preProcess = c("center", "scale"))
testData <- na.omit(testData)
predictValues <- predict(svm.model, newdata = testData)
confusionMatrix(predictValues, testData$income)
```

The output is as follows:

```
Confusion Matrix and Statistics

          Reference
Prediction <=50K  >50K
     <=50K 20634  3897
     >50K    635  1795

               Accuracy : 0.8319
                 95% CI : (0.8274, 0.8364)
    No Information Rate : 0.7889
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.3613

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9701
            Specificity : 0.3154
         Pos Pred Value : 0.8411
         Neg Pred Value : 0.7387
             Prevalence : 0.7889
         Detection Rate : 0.7653
   Detection Prevalence : 0.9099
      Balanced Accuracy : 0.6427

       'Positive' Class : <=50K
```

We note that the accuracy in this case is 0.8319, which is lesser than when we built the previous model with larger amount of data. In some situations, that could be considered a meaningful loss of performance, so we may have cut down our training data a little too much.

Now, let's try **k-fold cross validation** with a larger chunk of data used for training. For this dataset, which is quite large, we can use 70% of the data for training and holdout 30%. First, we can see the training method is now **repeated cross validation ("repeatedecv").** In the object that we use to control the training process, we defined k (the number of folds) to be 10. The algorithm will also use a process of model tuning (three different values of C: more on this later), so, the SVM algorithm will run 30 repetitions (10 folds times three values of C). Depending upon your computer, this could take a while.

```
#set up the same data as for the 1st example
set.seed(111)
#Create the test and train dataset
trainList <- createDataPartition(
    y = incomeDF$income, p=0.70, list = FALSE)
trainData <- incomeDF[trainList,]
testData <- incomeDF[-trainList,]
#Generate the SVM model
trctrl <- trainControl(method = "repeatedcv", number=10)
trainData <- na.omit(trainData)
svm.model.kfold <- train(income ~.,
                  data = trainData,
                  method= "svmRadial",
                  trControl = trctrl,
                  preProcess = c("center", "scale"))
svm.model.kfold
```

The output is as follows:

```
Support Vector Machines with Radial Basis Function Kernel

19886 samples
   14 predictor
    2 classes: '<=50K', '>50K'

Pre-processing: centered (36), scaled (36)
Resampling: Cross-Validated (10 fold, repeated 1 times)
Summary of sample sizes: 17897, 17897, 17898, 17897, 17897, 17897, ...
Resampling results across tuning parameters:

  C     Accuracy   Kappa
  0.25  0.8506991  0.4725875
  0.50  0.8527108  0.4868477
  1.00  0.8541189  0.4978871

Tuning parameter 'sigma' was held constant at a value of 0.02754888
Accuracy was used to select the optimal model using the largest value.
The final values used for the model were sigma = 0.02754888 and C = 1.
```

The accuracy for the final training model was 0.85, like the first model. If you are confused about how R calculated the accuracy (since the predictions with the test data have yet to be done), remember that when doing k-fold cross validation, for each repetition of the model, R holds out 1/k amount of data.

It is that hold out data that is used to calculate the accuracy for each of the iterations in the k-fold cross validation process. The final accuracy is the average of each of the iteration's validation accuracy calculations. We can also test the model on novel data and see if there is any shrinkage in model quality. Using the test Data to produce a new set of predictions, here's a look at the confusion matrix:

```
testData <- na.omit(testData)
predictValues <- predict(svm.model.kfold, newdata = testData)
confusionMatrix(predictValues, testData$income)
```

The output is as follows:

```
Confusion Matrix and Statistics

          Reference
Prediction <=50K >50K
     <=50K  6422  921
     >50K    273  831

               Accuracy : 0.8586
                 95% CI : (0.851, 0.866)
    No Information Rate : 0.7926
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.5021

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9592
            Specificity : 0.4743
         Pos Pred Value : 0.8746
         Neg Pred Value : 0.7527
             Prevalence : 0.7926
         Detection Rate : 0.7603
   Detection Prevalence : 0.8693
      Balanced Accuracy : 0.7168

       'Positive' Class : <=50K
```

The overall prediction accuracy in testData was essentially identical to accuracy from the k-fold cross-validated original model. When using k-fold cross validation for the training process we should not generally see much shrinkage when the model is applied to novel data.

We need to be aware of an important parameter **C (Cost of constraints)** in this case.

- If we specify a large value of C, we will get fewer mistakes. On the other hand,
- If we have a low value of C, we will get a more generalizable model, but one that makes more classification mistakes when we are building the model.

When using **k-fold cross validation**, a process known as **model tuning** is also performed. This is a process of identifying the best parameters for the machine learning algorithm. For SVM, model tuning is focused on identifying the best value of the cost parameter, C. When using the simple partitioning (train and test) approach, the model tuning needs to be done explicitly by the data scientist. But, when using k-fold, the training function does the tuning for you. By default, tuning is done three times, and that is why we get the accuracy of the model with three different values of C.

So, when using k-fold with 10 folds, we run the SVM machine learning algorithm 30 times (10 times for each of the C values evaluated), to create 30 models. Note that the **sigma parameter** — a value that controls overfitting was automatically selected for us. Thank goodness for that, because it would have taken a lot of experimentation to choose a reasonable value without the help of the tuning process.

Now, let's peek at what the "**support vectors**" are. Remember that the idea of a support vector is that it is an observation from the dataset that is close to the separator line or plane (or hyperplane) and therefore influential on the specific position of the line or plane. For each support vector, there is a weight, called **alpha,** that determines it "Strength" in influencing the position of the line or plane. Most of these alphas are zero, because many of the points in the dataset are far from the separator.

Now, we will look inside the training output object to find the alpha values. Because we are working with an advanced R object, we will use the "®" to access elements of the object. First, let's see the number of support vectors ("nSV"):

```
svm.model.kfold$finalModel@nSV
```

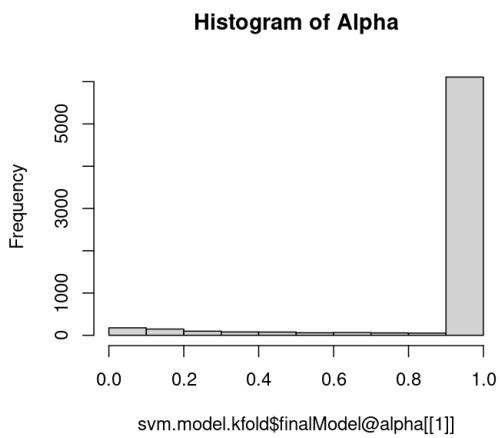
The output is as follows:

```
> svm.model.kfold$finalModel@nSV
[1] 6921
```

So, in our training data with 6921 of the observations are close to the separator. That also means that alpha is zero for the other observations, indicating that those observations were far enough away from the separator that they didn't have any influence on its position. Now we can review the list of alpha values for the actual support vectors. Let's make a histogram of the nonzero alphas with the following command:

```
hist(svm.model.kfold$finalModel@alpha[[1]],
     main = "Histogram of Alpha")
```

The output is as follows:



The histogram output reveals the range of the support vectors from 0 to 1. The maximum value of alpha is equal to the cost parameter that we discussed earlier. We can see that many of the support vectors are at this maximum alpha value. The support vectors at the maximum represent the observations that were right up against the separator.

Unfortunately, to get a handle on exactly how the SVM model works is tricky and does not provide full insight into how the prediction is made, which has led some data scientists to call SVM a "**black box**" whose inner workings are inscrutable.

Supervised Learning Via Classification and Regression Trees

While SVM can be very powerful, there is a fundamental challenge when using it: the algorithm is not transparent (black box) in terms of its internal workings. Using a black-box algorithm can be problematic because we may need to provide a clear explanation of how a prediction or classification process works for some audiences. This explanation might be needed for legal reasons (e.g., why did you decline a customer) or to help managers understand how the machine learning algorithm is doing the classification (e.g., how is this model working). In the examples below, we will explore a different machine learning approach that can provide a more intuitive explanation of how the model works.

Sometimes, these easier-to-understand models do not perform as well as the black-box methods. So, when building machine learning models, you need to think about whether model transparency is important for your project and whether you can tolerate somewhat lower accuracy in exchange for greater transparency.
In our new approach, **Classification and Regression Trees**, we ask a series of questions, and the answer to each subsequent question helps further reduce the uncertainty about which classification is most likely correct. Specifically, we will use the **rpart** algorithm (**Recursive Partitioning and Regression Trees**).

In our example, we will define a decision tree of rules that defines whether we think a person has income ≤50K or >50K. As can be seen below, the code for using **rpart** is very similar to the code we used to create an SVM model except the method is now "**rpart"** (rather than "**svmRadial**"). Notice that we are using the same training control as before (k-fold cross validation with k = 10). However, we did define one additional parameter. The **tuneLength** parameter tells the algorithm how many different default values to try for the main tuning parameter of the algorithm. So, rather than the default of three for the tuning length, we will have 50 (so, for each cross validation, we will try 50 different values for tuning).

```
model.rpart <- train(income ~ ., method = "rpart",
                     data = trainData,
                     trControl = trctrl,
                     tuneLength = 50)
model.rpart
```

The output is as follows:

```
CART

19886 samples
   14 predictor
    2 classes: '<=50K', '>50K'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 1 times)
Summary of sample sizes: 17896, 17898, 17898, 17898, 17898, 17896, ...
Resampling results across tuning parameters:

  cp           Accuracy   Kappa
  0.000000000  0.8453680  0.5159909
  0.001476017  0.8640253  0.5291706
  0.002952033  0.8621642  0.5225282
  0.004428050  0.8628179  0.5268587
  0.005904066  0.8599511  0.5119592
  0.007380083  0.8565313  0.5063638
  0.008856099  0.8539165  0.4914477
  0.010332116  0.8531123  0.4902058
  0.011808132  0.8514026  0.4721119
  0.013284149  0.8496426  0.4538721
  0.014760165  0.8496426  0.4538721
  0.016236182  0.8496426  0.4538721
  0.017712199  0.8489384  0.4475991
  0.019188215  0.8489384  0.4475991
  0.020664232  0.8489384  0.4475991
  0.022140248  0.8489384  0.4475991
  0.023616265  0.8489384  0.4475991
  0.025092281  0.8489384  0.4475991
  0.026568298  0.8489384  0.4475991
  0.028044314  0.8489384  0.4475991
  0.029520331  0.8489384  0.4475991
  0.030996347  0.8489384  0.4475991
  0.032472364  0.8489384  0.4475991
  0.033948380  0.8427535  0.4305029
  0.035424397  0.8401382  0.4175148
  0.036900414  0.8382286  0.4199539
  0.038376430  0.8382286  0.4199539
  0.039852447  0.8372739  0.4263080
  0.041328463  0.8366702  0.4337771
  0.042804480  0.8349606  0.4449088
  0.044280496  0.8349606  0.4449088
  0.045756513  0.8349606  0.4449088
  0.047232529  0.8345582  0.4525865
  0.048708546  0.8338036  0.4581909
  0.050184562  0.8338036  0.4581909
  0.051660579  0.8338036  0.4581909
  0.053136596  0.8338036  0.4581909
  0.054612612  0.8338036  0.4581909
  0.056088629  0.8338036  0.4581909
  0.057564645  0.8338036  0.4581909
  0.059040662  0.8338036  0.4581909
  0.060516678  0.8338036  0.4581909
  0.061992695  0.8338036  0.4581909
  0.063468711  0.8338036  0.4581909
  0.064944728  0.8338036  0.4581909
  0.066420744  0.8338036  0.4581909
  0.067896761  0.8338036  0.4581909
  0.069372778  0.8338036  0.4581909
  0.070848794  0.8338036  0.4581909
  0.072324811  0.8085044  0.2228175

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.001476017.
```

For each line, Caret was tuning "cp", which is the complexity parameter. The complexity parameter is an important tuning parameter for the **rpart** algorithm. The complexity parameter is used to control the number of trees in the final solution. If the cost of adding another branch to the decision tree from the current node is above the value of cp, then building that additional subtree does not continue. Of course, just like for the cost parameter "C" in SVM, having more trees will create a more accurate model for the training data, but might not be generalizable to other (i.e., new) data.

Now, let's see how this model does with our test data:

```
predictValues <- predict(model.rpart,
                         newdata = testData)
confusionMatrix(predictValues, testData$income)
```

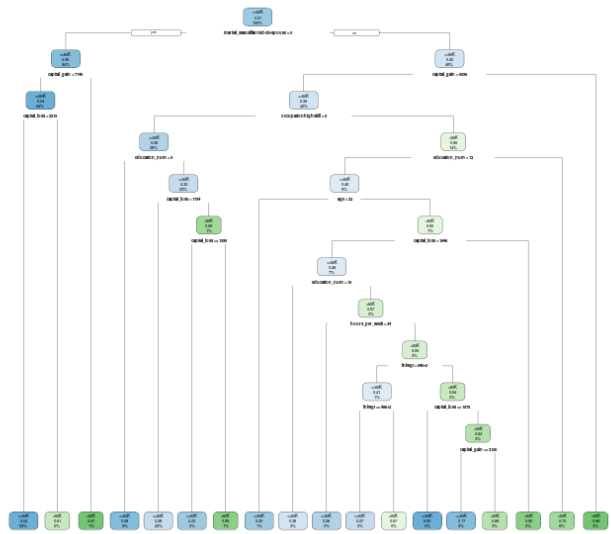
The output is as follows:

```
Confusion Matrix and Statistics

          Reference
Prediction <=50K >50K
     <=50K  6448  885
     >50K    247  867

               Accuracy : 0.866
                 95% CI : (0.8585, 0.8732)
    No Information Rate : 0.7926
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.5291

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9631
            Specificity : 0.4949
         Pos Pred Value : 0.8793
         Neg Pred Value : 0.7783
             Prevalence : 0.7926
         Detection Rate : 0.7633
   Detection Prevalence : 0.8681
      Balanced Accuracy : 0.7290

       'Positive' Class : <=50K
```

Because we are creating decision trees, we can visualize the trees directly:

```
install.packages("rpart.plot")
library(rpart.plot)
rpart.plot(model.rpart$finalModel)
```

The output is as follows:



That's hard to read without being able to zoom in, but the overall shape gives a flavor of how the model works. The top-level split is on the **marital status** and the second level splits include whether someone had capital gains (which would mean that they had investments that generated income), and education level. By the time we get to the bottom, we have a lots of "leaf" nodes that show whether someone should be classified as income ≤50K or >50K. When you run this code in R-studio, use the zoom button in the Plots window to explore the tree in greater detail.

We can also explore, which attributes were most important:

```
varImp(model.rpart)
```

The output is as follows:

```
rpart variable importance

  only 20 most important variables shown (out of 39)

                                     Overall
education_num                       100.0000
capital_gain                         99.6279
marital_statusMarried-civ-spouse     72.6483
age                                  62.6457
occupationhighskill                  46.6195
marital_statusNever-married          37.4996
capital_loss                         33.9422
educationgraduate                    26.1035
hours_per_week                       14.4930
educationmaster                       4.6312
fnlwgt                                2.0456
raceAsian-Pac-Islander                0.6042
raceWhite                             0.2608
sexMale                               0.0000
educationphd                          0.0000
`marital_statusMarried-spouse-absent`  0.0000
occupationunknown                     0.0000
`raceAsian-Pac-Islander`              0.0000
relationshipUnmarried                 0.0000
occupationagricultr                   0.0000
```

We could stop here, except for one important issue: is it appropriate to use all these attributes?

Because this model could be used within the banking industry to decide which people should get more credit, it would be highly inappropriate (and in some places, illegal) to use the sex, age, and race attributes within our model. So, let's remove those attributes from our dataset and then rerun the analysis:

```
trainData <- dplyr::select(trainData, -sex, -age, -race)
rpart.model.new <- train(income ~ .,
                    method = "rpart",
                    trControl = trctrl,
                    data = trainData, tuneLength = 50)
predictValues <- predict(rpart.model.new,
                         newdata = testData)
cm <- confusionMatrix(predictValues, testData$income)
cm$overall[1]
```

The output is as follows:

```
 Accuracy
0.8649225
```

So, using **rpart** and trimming out variables that would not be ethical to use in the loan or credit decision, we have a model that is explainable and nearly as accurate as the best model we produced.

Important R Functions

- **as.factor( ):** Create an object, particularly a string, to a factor variable
- **confusionMatrix( ):** Generates a confusion matrix for predicted and actual values.
- **createDataPartition( ):** Chooses elements at random from a vector
- **filter( ):** Tidyverse function to subset rows using column values
- **list( ):** Create an R list object
- **mutate_if( ):** Tidyverse function to change a column for some cases.
- **predict( ):** Uses a model to predict an outcome.
- **select( ):** Tidyverse function to choose a subset of rows.
- **table( ):** Creates tabular results for a given list of categorical variables.
- **train( ):** Generic caret function to run training procedure.